フォトアルバムオーガナイザー憲法

核となる 5 つの原則

I. ファイル駆動開発の厳格実施

非交渉型要件（MUST）：

- すべての機能実装は、テスト作成 → ユーザー承認 → テスト失敗確認 → 実装 → テスト成功 の Red-Green-Refactor サイクルに従う必須
- 受け入れシナリオはすべてテストケースに変換し、実装前にテストが失敗することを確認する必須
- 単体テスト、統合テスト、コントラクトテストの 3 層テスト戦略を実装する必須
- 前機能品質、PR は MIME タイプ、ファイルサイズ、ファイル拡張子の 3 層検証を実装する必須

根拠: ユーザーストーリーを仕様に利用して検証可能にし、グレード防止と品質保証を実現する。テスト失敗から始めることで、要件漏れ検出の精度を高める。

II. セキュリティ要件の機能要件優先化

非交渉型要件（MUST）：

- 機密データ（ユーザメタデータ、タイムスタンプ、ファイル名）の平文保存を禁止。暗号化または(ハッシュ)化を必須とする
- 外部サーバへのデータ送信、ロギング、分析トラッキングを禁止。ローカルストレージのみでのデータ処理を必須とする
- ファイルアップロード時は MIME タイプ、ファイルサイズ、ファイル拡張子の 3 層検証を実装する必須
- セキュリティ関連のコード変更は、セキュリティレビュー承認が機能要件承認に先行する必須

根拠: ユーザの写真とメタデータはプライベート資産。外部漏洩リスク排除が事業継続の前提条件。機能要件より優先することで、初期段階からセキュリティ脆弱性を排除する。

III. パフォーマンス閾値の定量化と受け入れ基準統合

非交渉型要件（MUST）：

- UI 応答時間は 1 秒以下を上限とし、メインスレッドブロッキング時間を 100ms 以下に制限する必須
- JavaScript バンドルサイズは 300KB 以下を上限、超過時は機能削減または動的読み込みを実装する必須
- メモリ使用量は 200MB 以下を上限、100 枚以上の写真処理時もこの制限内に収まる必須
- パフォーマンス閾値の違反検知は、各フェーズの受け入れ基準に組み込み、未達成時は本番リリース不可とする必須

根拠: モバイルデバイスでの動作保証とユーザー満足度維持。定量化することで、パフォーマンス劣化を早期に検知でき、後続フェーズで対応コストが増加する問題を回避できる。

IV. ユーザー体験の一貫性維持

非交渉型要件（MUST）：

- UI コンポーネントは統一されたデザイン言語を採用し、画面遷移はすべて一貫したフローを保つ必須
- エラーメッセージはユーザーに対して原因と対応方法を明示し、技術用語を避ける必須
- すべての入力フォームに対し、バリデーションエラー時のフィードバック（リアルタイム表示 ≤ 200ms）を提供する必須
- ダークモード / ライトモード、アクセシビリティ（WCAG 2.1 Level AA）対応を実装する必須

根拠: 一貫したユーザー体験は、機能の学習コストを低減し、アプリケーション全体の信頼性を向上させる。モバイル・タブレット・デスクトップ間での動作一貫性も確保する。

V. コード品質と保守性の継続的改善

非交渉型要件（MUST）：

- すべてのコードは ESLint + Prettier でのフォーマット統一を実装する必須
- 複雑度の高い関数（循環的複雑度 > 15）は、テスト + ドキュメント + レビュー承認を必須とする
- 外部依存はバージョンを固定し、`package-lock.json` を Git に含める必須。再現性を確保する必須
- コード変更時は、既存テストの全実行と新規テストの追加が必須。カバレッジ低下は本番リリース不可とする必須

根拠: コード品質の継続的維持は、機能追加時のグレード防止と保守コスト削減を実現する。バージョン固定による再現性は、本番環境での予期しない動作防止に不可欠。

---

3 つの制約条件

制約 I. 機密データの暗号化・ハッシュ化必須

内容:

- ユーザのメタデータ（撮影日時、写真の場所、カメラモデル、ファイルパス等）は、localStorage に平文で保存することを禁止
- 最小限、ハッシュ化（SHA-256）またはクライアント側暗号化（AES-GCM）を実装する必須
- 暗号化鍵の管理方法を明記し、鍵漏洩時の対応手順を事前に定義する必須

例外処理: なし

制約 II. 外部依存のバージョン固定による再現性確保

内容:

- `package.json` に記載するすべての依存は `~` または `=` ではなく、明示的にバージョン固定する（例: `"sqlj.js": "1.13.0"`）
- `npm ci` コマンドを使用し、`package-lock.json` から正確に同一バージョンを再現する必須
- 依存アップデートは計画的に行い、テスト + レビュー承認後にのみ本番適用する必須

例外処理: セキュリティ脆弱性修正時は、最小限の更新を実施後、直ちに本番適用可能

制約 III. 仕様と実装の乖離検知・是正

内容:

- 仕様書（`spec.md`）とコード実装は、毎フェーズ終了時に整合性チェックを実施する必須
- 乖離発見時は、レビュー会議で原因を特定し、仕様修正または実装修正のいずれかを選択して記録する必須
- 重大な乖離（受け入れ基準未達成等）は、本番リリース前に必ず是正する必須

例外処理: なし

---

3 つのガバナンスルール

ルール I. 開発の段階的ゲーティング

作業順序（必須流れ）：

1. **要件チェック**: 要件の 3 原則遵守確認（新規フィーチャー開始時）
2. **仕様策定**: `spec.md` でユーザーストーリー定義（Phase 0）
3. **計画策定**: `plan.md` で技術設計・リサーチ（Phase 0）
4. **タスク分解**: `tasks.md` で実装タスク化（Phase 1）
5. **実装検証**: テスト → 実装 → レビュー（Phase 2-N）
6. **統合テスト**: 各フェーズの受け入れ基準検証（各フェーズ末）
7. **リリース審査**: 本番環境デプロイ前の最終チェック

ゲート不合格時の対応: 次フェーズは進行禁止。前フェーズへ戻り修正。

ルール II. Git ブランチ戦略による仕様・実装分離

ブランチ命名規則:

```
mainブランチ（本番環境の正式版）
  ↑ (git checkout main; git checkout -b <番号>-<短い名前>)
仕様ブランチ: 001-photo-album-organizer, 002-batch-processing など
  ↑ (git checkout 001-photo-album-organizer; git checkout -b feature/impl-<番号>-<短い名前>)
実装ブランチ: feature/impl-001-metadata-encryption, feature/impl-001-ui-responsive など
  ↑ (Full Request + Review)
仕様ブランチにマージ
  ↑ (全ユーザーストーリー完了後、Pull Request + 最終レビュー)
mainにマージ
```

ブランチ対応表:

ブランチタイプ	派生元	目的	マージ先
仕様ブランチ	main	ユーザーストーリー仕様 + 技術計画 + 実装	main
実装ブランチ	仕様ブランチ	個別タスク実装	仕様ブランチ

コミットメッセージ形式:

- 仕様ブランチ: `docs: <フィーチャー> 仕様追加（<番号>）`
- 実装ブランチ: `feat: <機能> 実装（<番号>）` or `test: <テスト> 追加（<番号>）`
- テンプレート更新: `docs: テンプレート更新 - 憲法v1.0対応`

ルール III. 重大変更のレビュー承認必須化

重大変更の定義:

- **仕様変更**: ユーザーストーリーの受け入れ基準、セキュリティ要件の追加・削除・変更
- **パフォーマンス基準変更**: 応答時間、バンドルサイズ、メモリ上限の変更
- **依存ライブラリ**: メジャーバージョンアップ、セキュリティ脆弱性対応
- **API コントラクト変更**: 入出力形式、データ構造、エラーハンドリング

レビュープロセス:

1. Pull Request 作成時に「重大変更」ラベルを付与
2. チームメンバー 2 名以上の承認を取得する必須
3. 承認者は以下をチェック: 仕様/実装整合性、セキュリティリスク、パフォーマンス影響
4. 承認後、仕様ブランチにマージ

例外: なし（すべての重大変更は承認必須）

Git ブランチ戦略（詳細）

初期化フロー

```
# 1. メインブランチから仕様ブランチを作成
git checkout main
git checkout -b 001-photo-album-organizer

# 2. 仕様ブランチ上で計画・仕様・調査ドキュメントを作成・更新
#   (spec.md, plan.md, research.md, data-model.md, quickstart.md, contracts/*)
vim specs/001-photo-album-organizer/spec.md
git add specs/001-photo-album-organizer/
git commit -m "docs: photo-album-organizer 仕様書追加（001）"

# 3. 仕様ブランチから実装ブランチを作成
git checkout 001-photo-album-organizer
git checkout -b feature/impl-001-metadata-encryption

# 4. 実装実施、テスト作成、実装コード作成
vim src/utils/encryption.js
vim tests/unit/encryption.test.js
git add src/ tests/
git commit -m "feat: メタデータ暗号化実装（001）"

# 5. 実装ブランチから仕様ブランチへ Pull Request → Review → Merge
git push origin feature/impl-001-metadata-encryption
# GitHub で Pull Request 作成 → Review → Merge to 001-photo-album-organizer

# 6. 次の実装ブランチを作成（手順 1-5 を繰り返す）
git checkout 001-photo-album-organizer
git checkout -b feature/impl-001-ui-responsive
# ...実装...
git commit -m "feat: レスポンシブ UI 実装（001）"
git push origin feature/impl-001-ui-responsive
# GitHub で PR → Review → Merge

# 7. すべての実装ブランチ完了後、仕様ブランチから main へ
git checkout 001-photo-album-organizer
git pull origin 001-photo-album-organizer  # 最新同期
git checkout main
git merge 001-photo-album-organizer --no-ff -m "merge: photo-album-organizer フィーチャー完了（v1.0.0）"
git tag -a v1.0.0
git push origin main --tags
```

継続的開発フロー

```
# 新規フィーチャー追加時
git checkout main
git checkout -b 002-batch-processing

# その後の流れは「初期化フロー」の手順 2 以降と同一
```

Governance（ガバナンス）

憲法の修正プロセス

- **軽微な修正**（文言修正、タイポ）: パッチバージョンアップ, PR 1 名承認で可
- **原則追加・修正**（新しい原則の追加、既存原則の再定義）: マイナーバージョンアップ, PR 2 名承認必須, チーム全体に通知
- **原則削除・本質的変更**: メジャーバージョンアップ, 全チーム承認必須、移行計画作成必須

テンプレートの整合性確認

このファイルに記載された原則・制約・ガバナンスルールは、以下のテンプレートに反映されるべき：

- `.specify/templates/plan-template.md`:「Constitution Check」セクションでこの原則を参照
- `.specify/templates/spec-template.md`:「User Scenarios & Testing」で TDD 原則と優先度付け（P1-P3）ルールを適用
- `.specify/templates/tasks-template.md`: Phase 別タスクを分離でお独立的デーティングを実装
- `.specify/templates/checklist-template.md`: レビューチェックリストでセキュリティ優先・パフォーマンス基準確認を追加

準拠の技証方法

- 各 PR マージ前に、このファイルの原則・制約を PR 説明に記載する必須
- レビュアーは「憲法準拠」チェックボックスを確認してから承認する必須
- 本番リリース前に、すべての受け入れ基準（パフォーマンス・セキュリティ・テストカバレッジ）を再検証する必須

Version: 1.0.0 | Ratified: 2025-11-18 | Last Amended: 2025-11-18